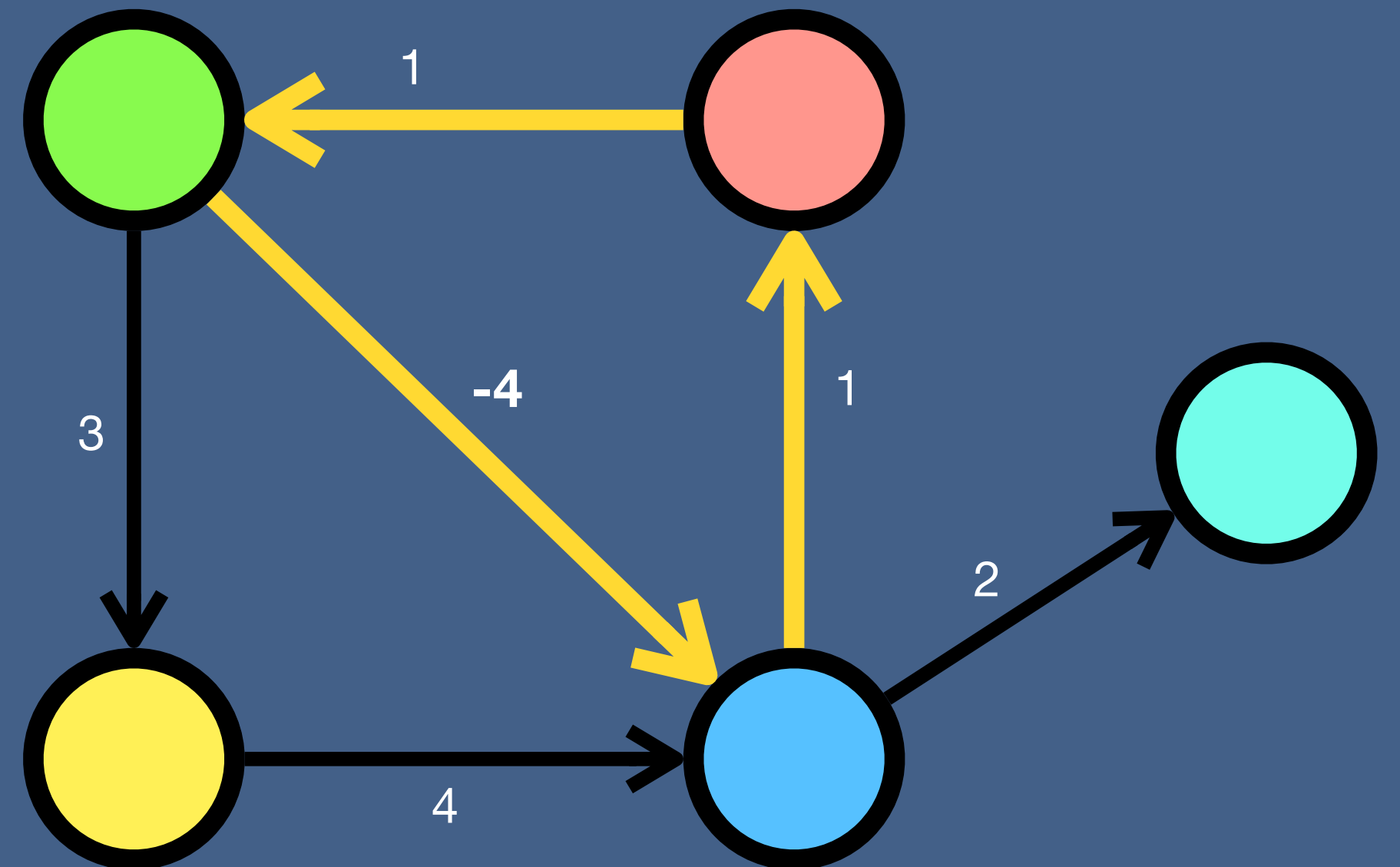


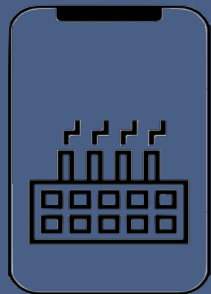
Bellman Ford Algorithm



Bellman Ford Algorithm

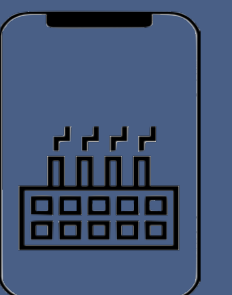
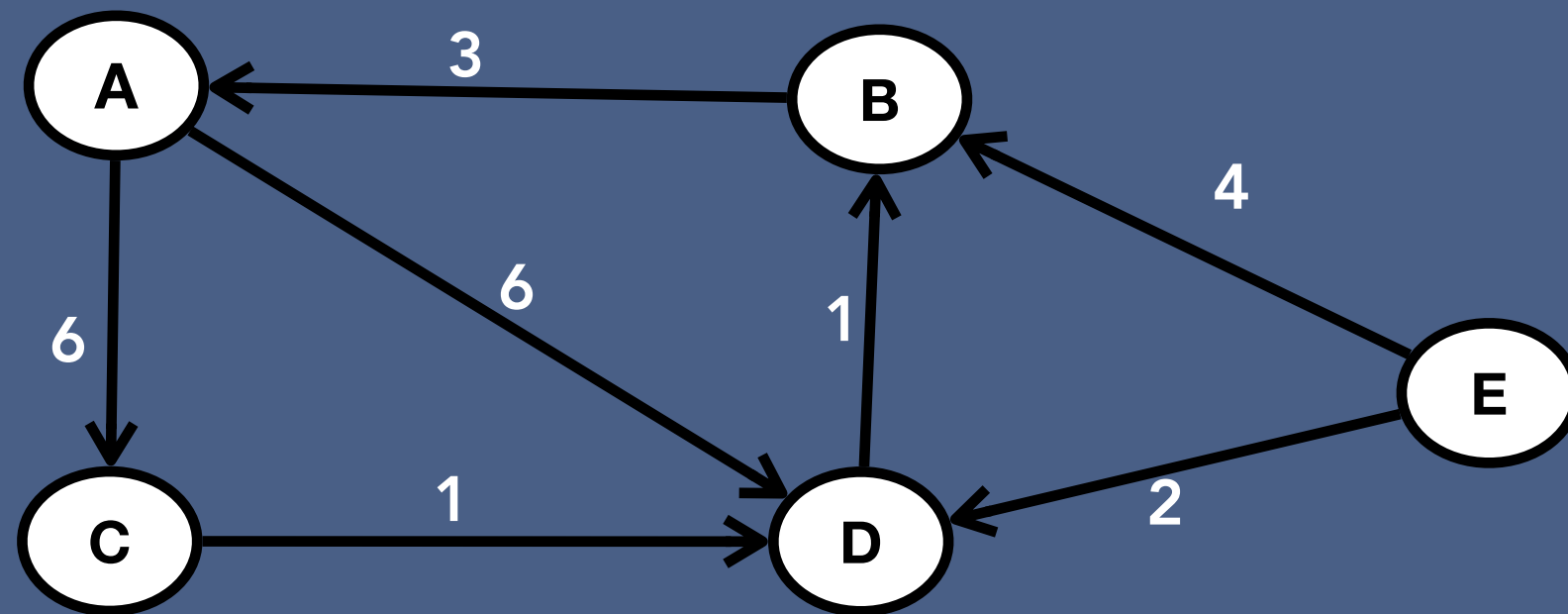
Single Source Shortest Path Algorithm

Graph Type	BFS	Dijkstra	Bellman Ford
Unweighted - undirected	OK	OK	OK
Unweighted - directed	OK	OK	OK
Positive - weighted - undirected	X	OK	OK
Positive - weighted - directed	X	OK	OK
Negative - weighted - undirected	X	OK	OK
Negative - weighted - directed	X	OK	OK
Negative Cycle	X	X	OK

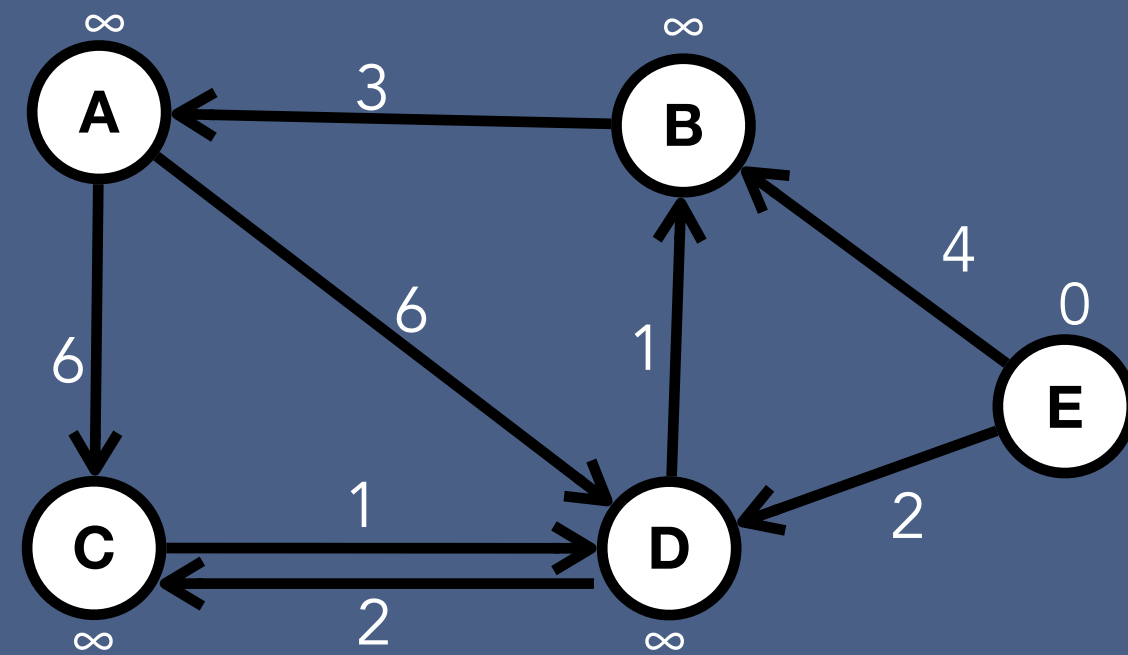


Bellman Ford Algorithm

Bellman Ford algorithm is used to find single source shortest path problem. If there is a negative cycle it catches it and report its existence.



Bellman Ford Algorithm

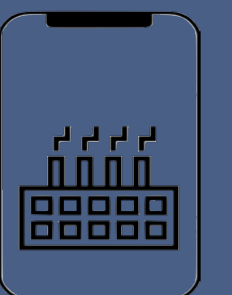


If the distance of destination vertex > (distance of source vertex + weight between source and destination vertex):

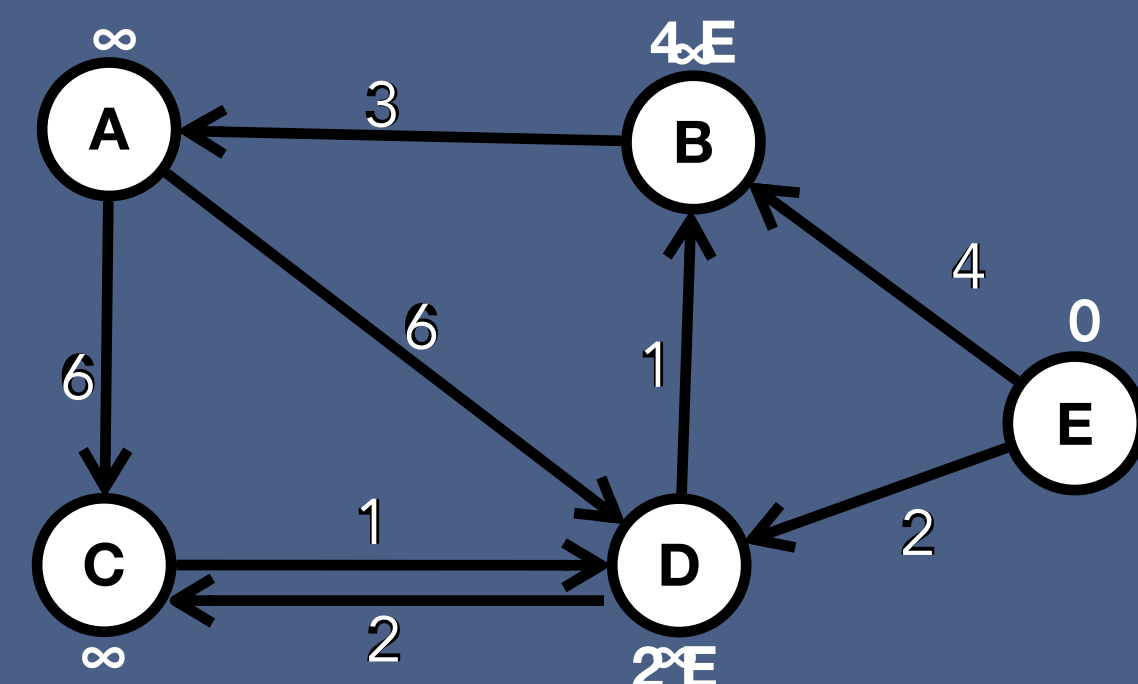
Update distance of destination vertex to (distance of source vertex + weight between source and destination vertex)

Edge	Weight
A->C	6
A->D	6
B->A	3
C->D	1
D->C	2
D->B	1
E->B	4
E->D	2

Distance Matrix	
Vertex	Distance
A	∞
B	∞
C	∞
D	∞
E	0



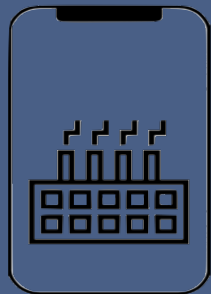
Bellman Ford Algorithm



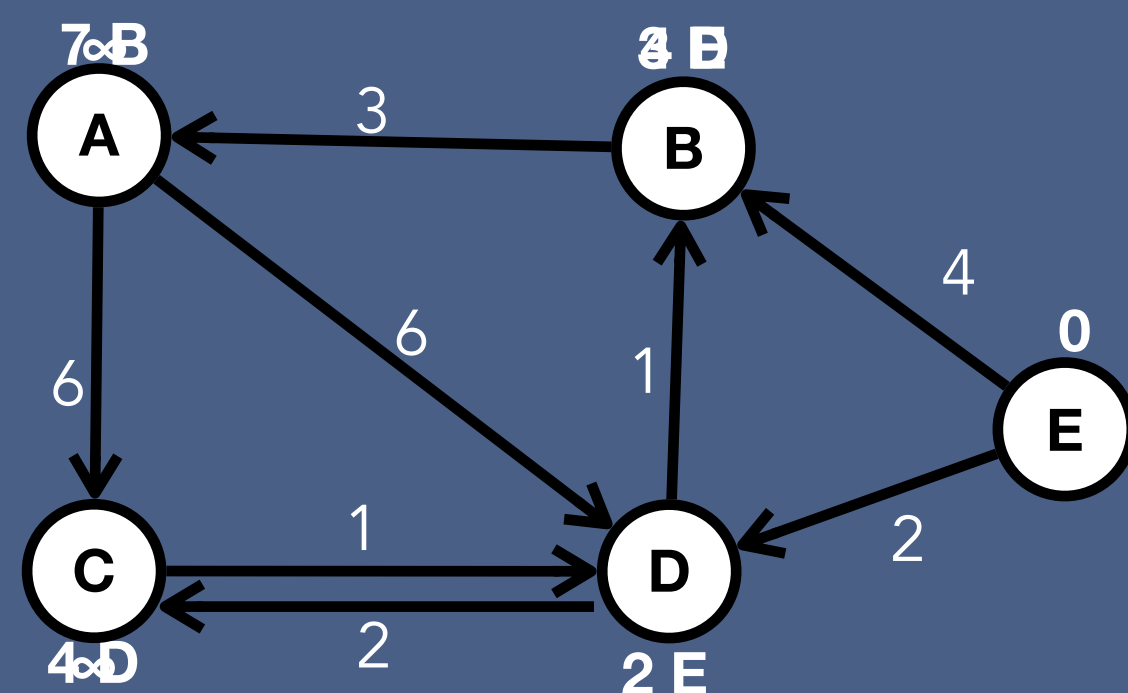
If the distance of destination vertex > (distance of source vertex + weight between source and destination vertex):
Update distance of destination vertex to (distance of source vertex + weight between source and destination vertex)

Edge	Weight
A->C	6
A->D	6
B->A	3
C->D	1
D->C	2
D->B	1
E->B	4
E->D	2

Distance Matrix		Iteration 1	
Vertex	Distance	Distance	Parent
A	∞		-
B	∞		E
C	∞		-
D	∞		E
E	0		-



Bellman Ford Algorithm

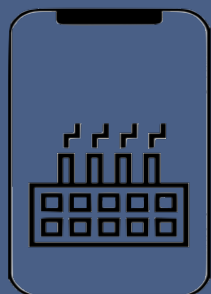


If the distance of destination vertex > (distance of source vertex + weight between source and destination vertex):

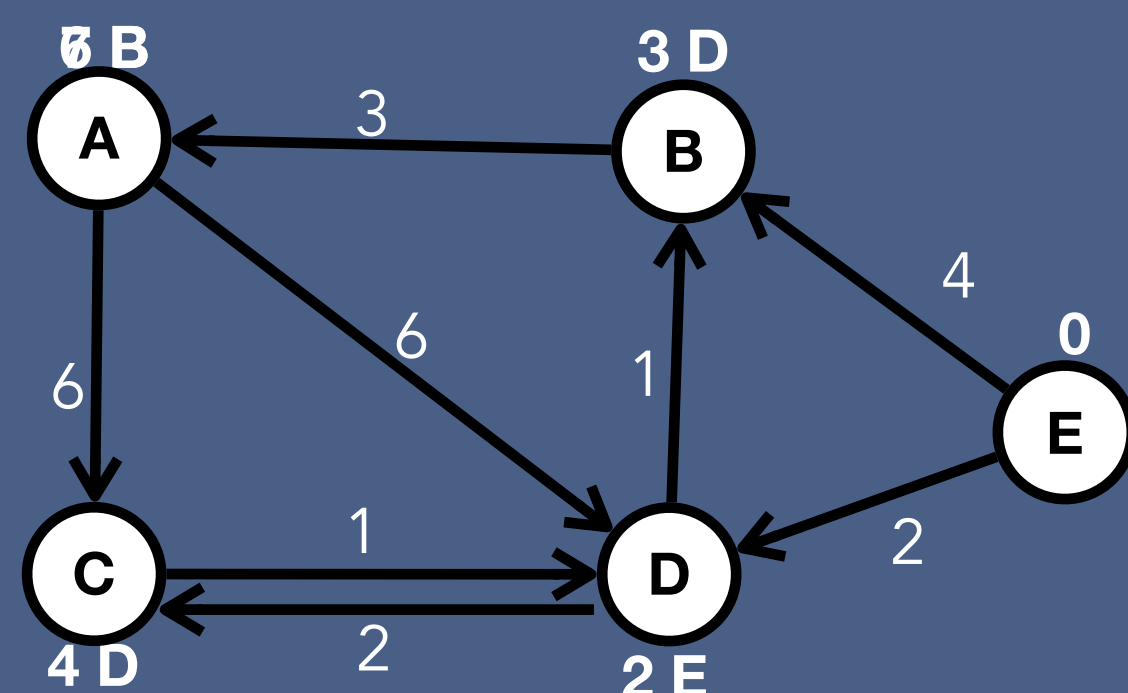
Update distance of destination vertex to (distance of source vertex + weight between source and destination vertex)

Edge	Weight
A->C	6
A->D	6
B->A	3
C->D	1
D->C	2
D->B	1
E->B	4
E->D	2

Distance Matrix					
Vertex	Distance	Iteration 1		Iteration 2	
		Distance	Parent	Distance	Parent
A	∞	∞	-	4+3=7	B
B	∞	4	E	2+1=3	D
C	∞	∞	-	2+2=4	D
D	∞	2	E	2	E
E	0	0	-	0	-



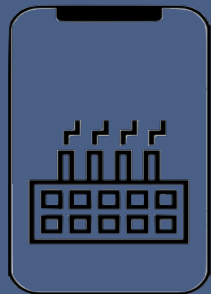
Bellman Ford Algorithm



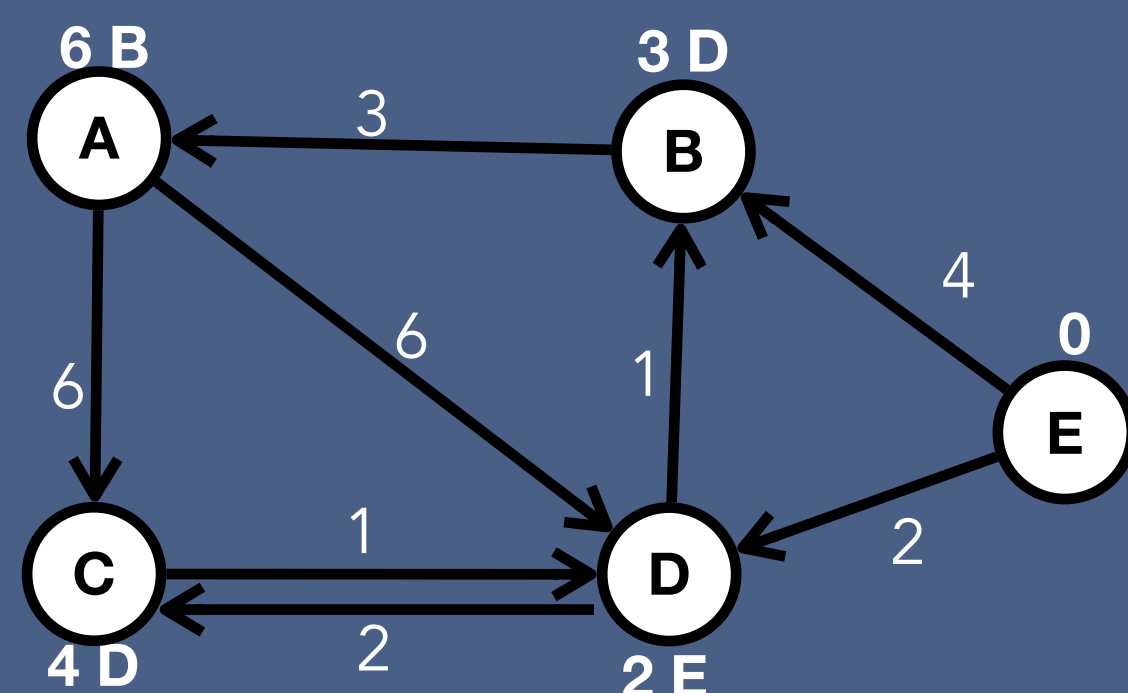
If the distance of destination vertex > (distance of source vertex + weight between source and destination vertex):
Update distance of destination vertex to (distance of source vertex + weight between source and destination vertex)

Edge	Weight
A->C	6
A->D	6
B->A	3
C->D	1
D->C	2
D->B	1
E->B	4
E->D	2

		Distance Matrix					
Vertex	Distance	Iteration 1		Iteration 2		Iteration 3	
		Distance	Parent	Distance	Parent	Distance	Parent
A	∞	∞	-	4+3=7	B	3+3=6	B
B	∞	4	E	2+1=3	D	3	D
C	∞	∞	-	2+2=4	D	4	D
D	∞	2	E	2	E	2	E
E	0	0	-	0	-	0	



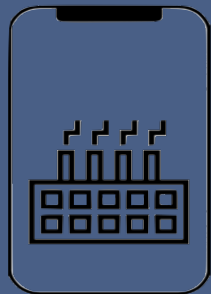
Bellman Ford Algorithm



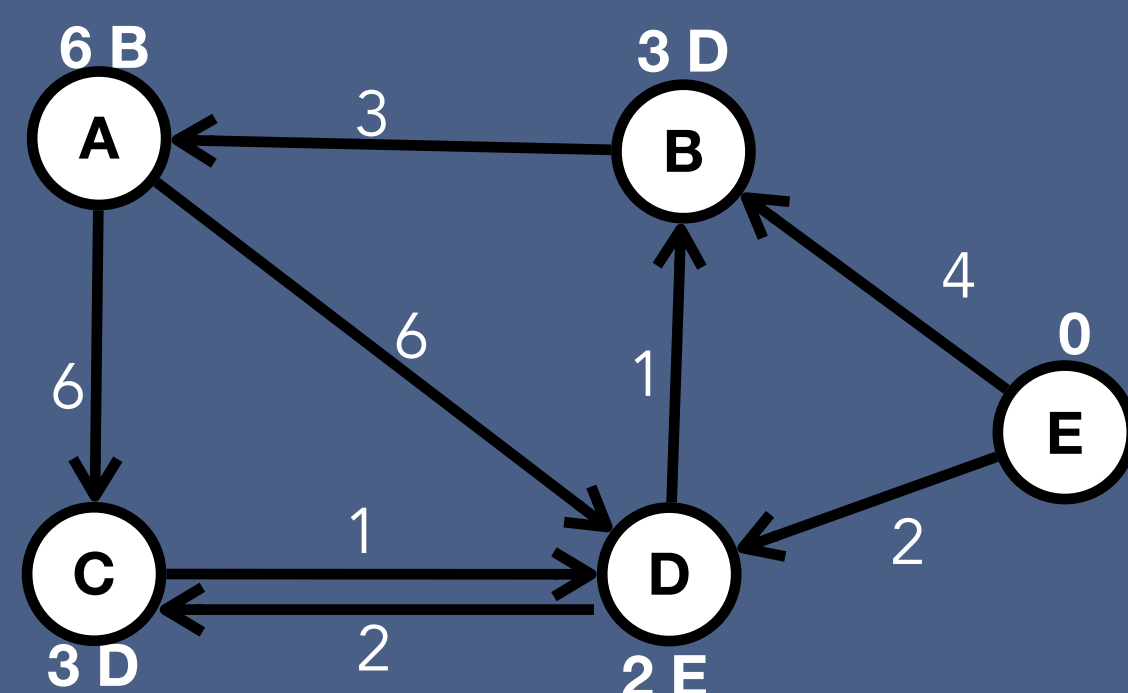
If the distance of destination vertex > (distance of source vertex + weight between source and destination vertex):
Update distance of destination vertex to (distance of source vertex + weight between source and destination vertex)

Edge	Weight
A->C	6
A->D	6
B->A	3
C->D	1
D->C	2
D->B	1
E->B	4
E->D	2

Distance Matrix									
Vertex	Distance	Iteration 1		Iteration 2		Iteration 3		Iteration 4	
		Distance	Parent	Distance	Parent	Distance	Parent	Distance	Parent
A	∞	∞	-	4+3=7	B	3+3=6	B	6	B
B	∞	4	E	2+1=3	D	3	D	3	D
C	∞	∞	-	2+2=4	D	4	D	4	D
D	∞	2	E	2	E	2	E	2	E
E	0	0	-	0	-	0	-	0	-



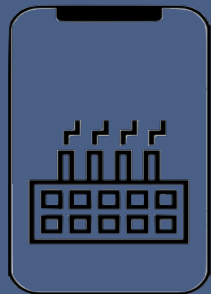
Bellman Ford Algorithm



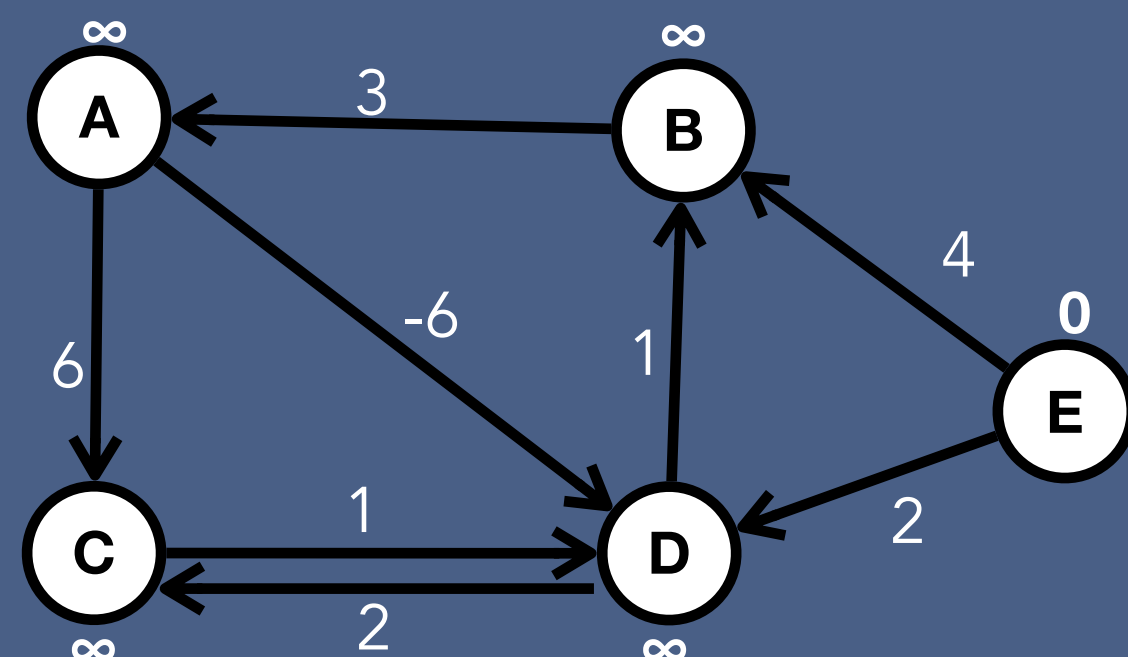
If the distance of destination vertex > (distance of source vertex + weight between source and destination vertex):
Update distance of destination vertex to (distance of source vertex + weight between source and destination vertex)

Edge	Weight
A->C	6
A->D	6
B->A	3
C->D	1
D->C	2
D->B	1
E->B	4
E->D	2

Final Solution		
Vertex	Distance from E	Path from E
A	6	E -> D -> B -> A
B	3	E -> D -> B
C	4	E -> D -> C
D	2	E -> D
E	0	0



Bellman Ford with Negative Cycle

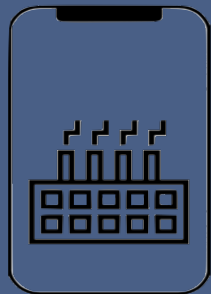


If the distance of destination vertex > (distance of source vertex + weight between source and destination vertex) :

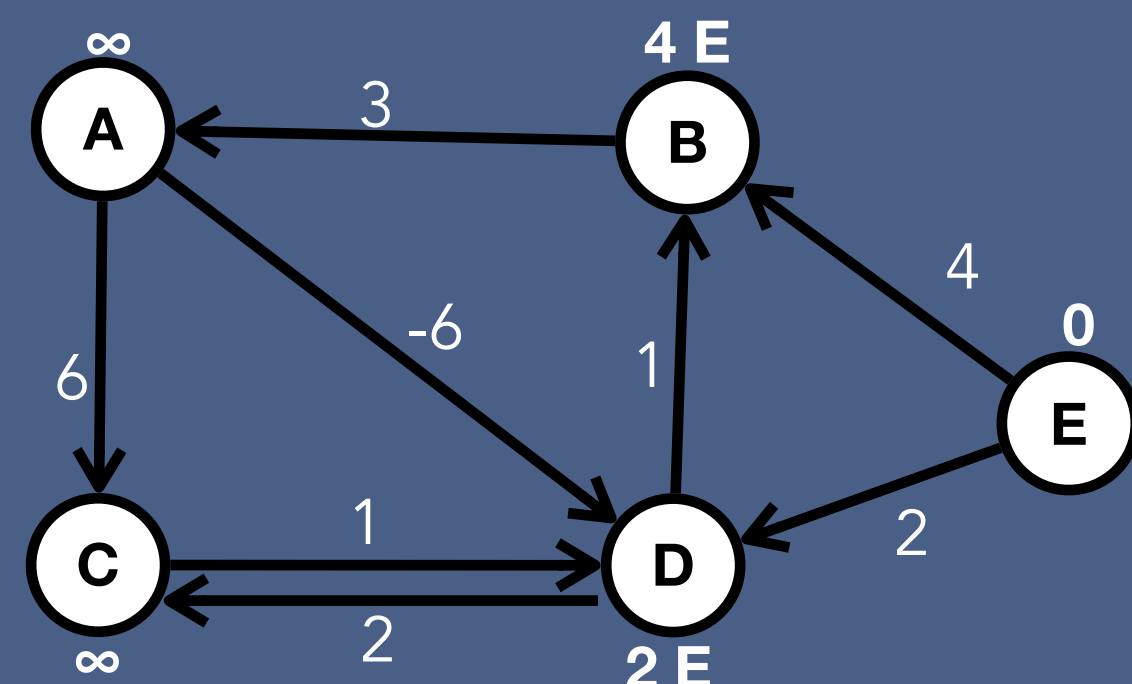
Update distance of destination vertex to (distance of source vertex + weight between source and destination vertex)

Edge	Weight
A->C	6
A->D	-6
B->A	3
C->D	1
D->C	2
D->B	1
E->B	4
E->D	2

Distance Matrix	
Vertex	Distance
A	∞
B	∞
C	∞
D	∞
E	0



Bellman Ford with Negative Cycle

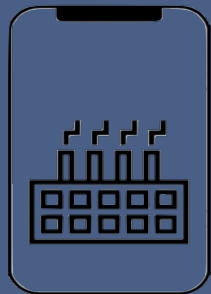


If the distance of destination vertex > (distance of source vertex + weight between source and destination vertex):

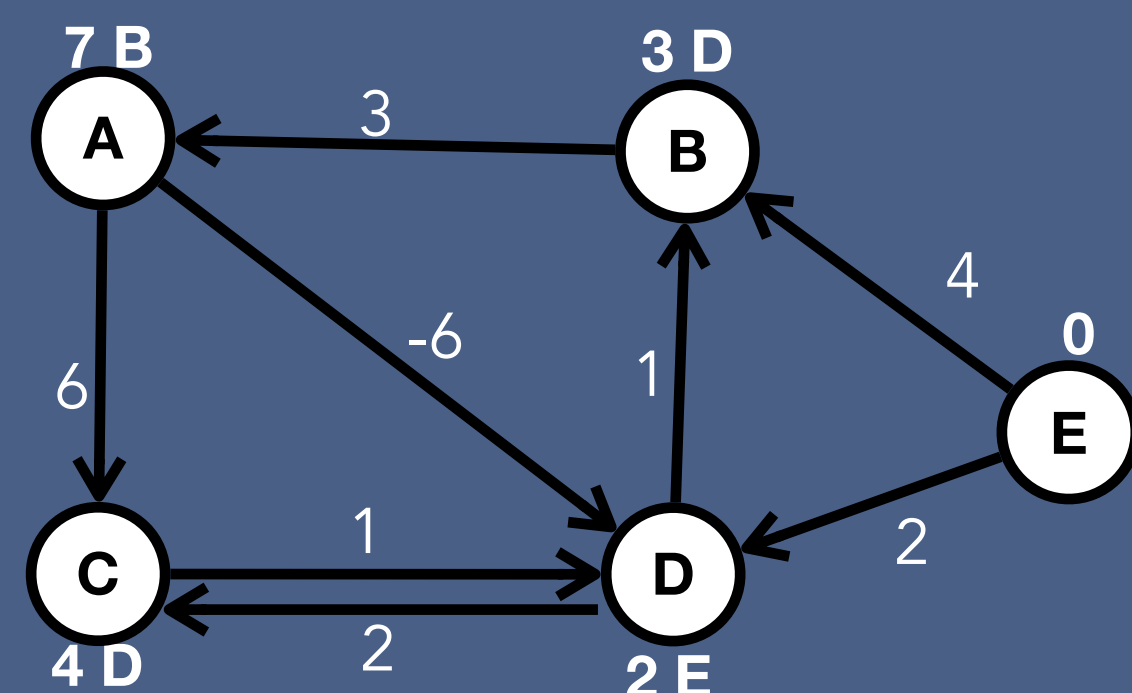
Update distance of destination vertex to (distance of source vertex + weight between source and destination vertex)

Edge	Weight
A->C	6
A->D	-6
B->A	3
C->D	1
D->C	2
D->B	1
E->B	4
E->D	2

Distance Matrix			
Vertex	Distance	Iteration 1	
		Distance	Parent
A	∞	∞	-
B	∞	4	E
C	∞	∞	-
D	∞	2	E
E	0	0	-



Bellman Ford with Negative Cycle

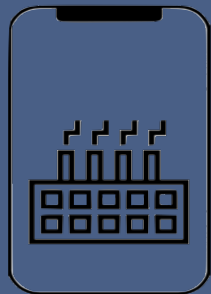


If the distance of destination vertex > (distance of source vertex + weight between source and destination vertex):

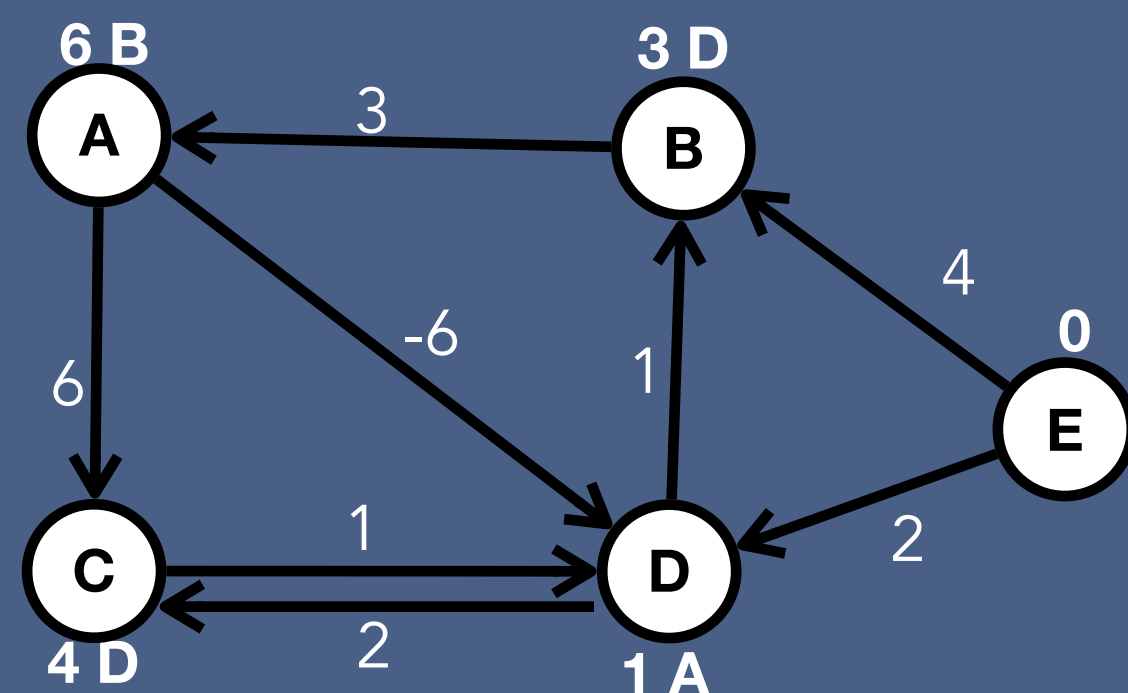
Update distance of destination vertex to (distance of source vertex + weight between source and destination vertex)

Edge	Weight
A->C	6
A->D	-6
B->A	3
C->D	1
D->C	2
D->B	1
E->B	4
E->D	2

Distance Matrix					
Vertex	Distance	Iteration 1		Iteration 2	
		Distance	Parent	Distance	Parent
A	∞	∞	-	4+3=7	B
B	∞	4	E	2+1=3	D
C	∞	∞	-	2+2=4	D
D	∞	2	E	2	E
E	0	0	-	0	-



Bellman Ford with Negative Cycle

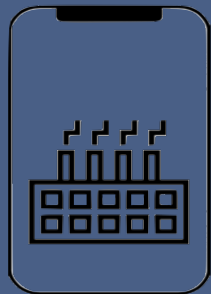


If the distance of destination vertex > (distance of source vertex + weight between source and destination vertex):

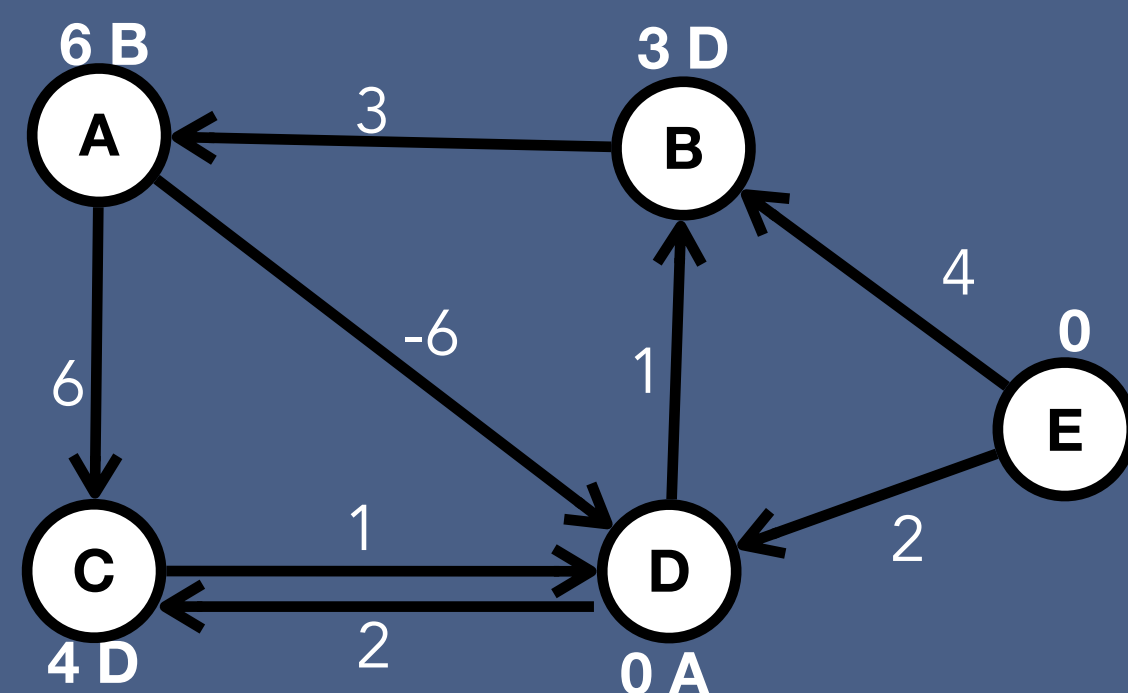
Update distance of destination vertex to (distance of source vertex + weight between source and destination vertex)

Edge	Weight
A->C	6
A->D	-6
B->A	3
C->D	1
D->C	2
D->B	1
E->B	4
E->D	2

Distance Matrix							
Vertex	Distance	Iteration 1		Iteration 2		Iteration 3	
		Distance	Parent	Distance	Parent	Distance	Parent
A	∞	∞	-	4+3=7	B	3+3=6	B
B	∞	4	E	2+1=3	D	3	D
C	∞	∞	-	2+2=4	D	4	D
D	∞	2	E	2	E	7+(-6)=1	A
E	0	0	-	0	-	0	



Bellman Ford with Negative Cycle

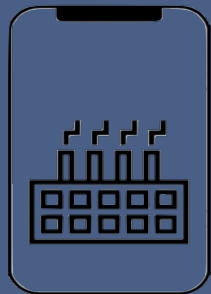


If the distance of destination vertex > (distance of source vertex + weight between source and destination vertex):

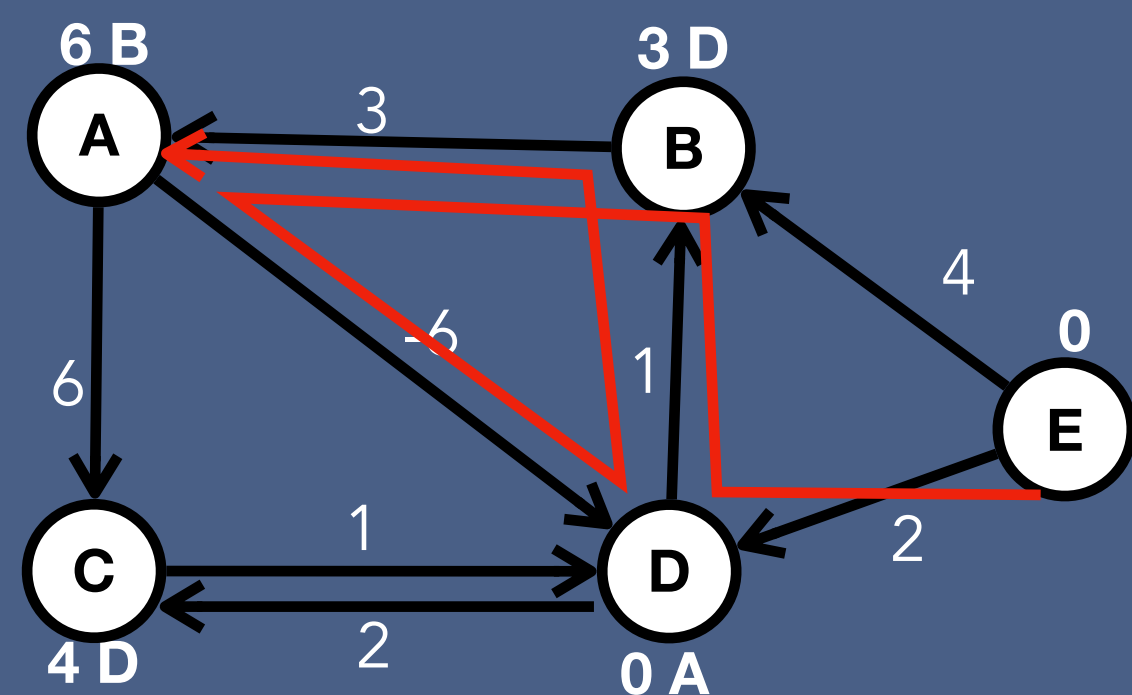
Update distance of destination vertex to (distance of source vertex + weight between source and destination vertex)

Edge	Weight
A->C	6
A->D	-6
B->A	3
C->D	1
D->C	2
D->B	1
E->B	4
E->D	2

Distance Matrix									
Vertex	Distance	Iteration 1		Iteration 2		Iteration 3		Iteration 4	
		Distance	Parent	Distance	Parent	Distance	Parent	Distance	Parent
A	∞	∞	-	4+3=7	B	3+3=6	B	6	B
B	∞	4	E	2+1=3	D	3	D	3	D
C	∞	∞	-	2+2=4	D	4	D	4	D
D	∞	2	E	2	E	7+(-6)=1	A	6+(-6)=0	A
E	0	0	-	0	-	0	-	0	-



Bellman Ford with Negative Cycle

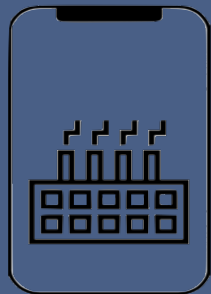


If the distance of destination vertex > (distance of source vertex + weight between source and destination vertex):

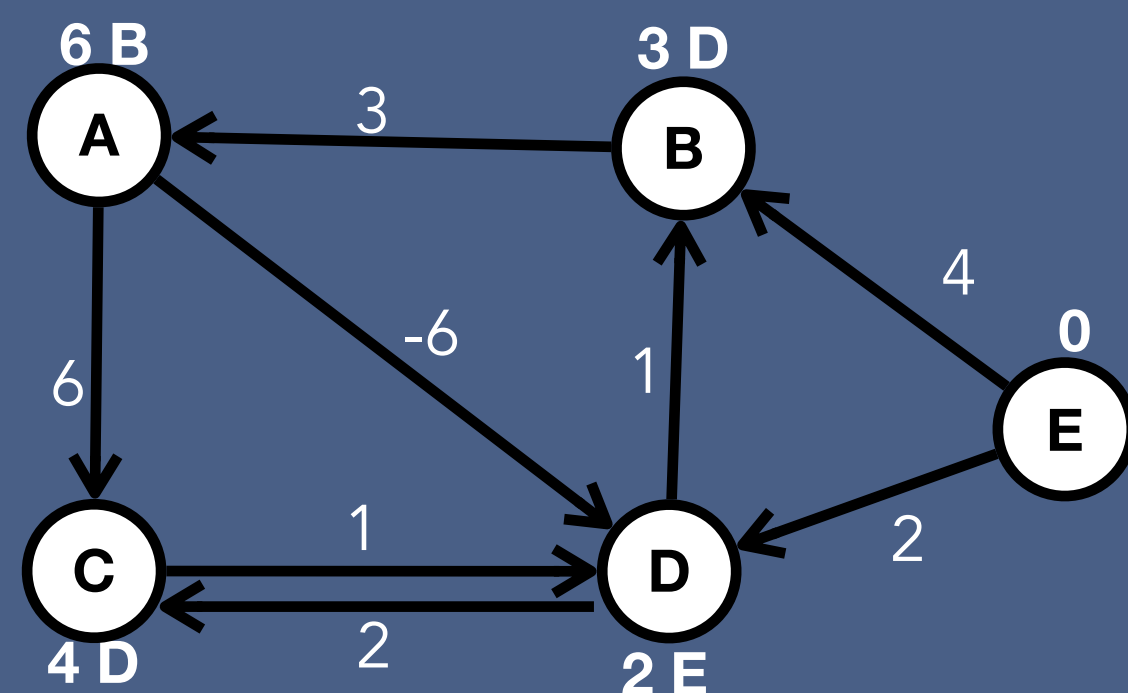
Update distance of destination vertex to (distance of source vertex + weight between source and destination vertex)

Distance Matrix											
Vertex	Distance	Iteration 1		Iteration 2		Iteration 3		Iteration 4		Iteration 5	
		Distance	Parent	Distance	Parent	Distance	Parent	Distance	Parent	Distance	Parent
A	∞	∞	-	$4+3=7$	B	$3+3=6$	B	6	B	4	B
B	∞	4	E	$2+1=3$	D	3	D	3	D	0	D
C	∞	∞	-	$2+2=4$	D	4	D	4	D	0	D
D	∞	2	E	2	E	$7+(-6)=1$	A	$6+(-6)=0$	A	1	A
E	0	0	-	0	-	0	-	0	-		

Edge	Weight
A->C	6
A->D	-6
B->A	3
C->D	1
D->C	2
D->B	1
E->B	4
E->D	2



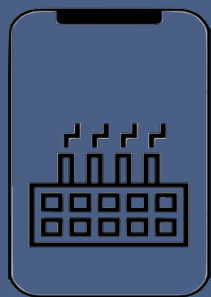
Bellman Ford with Negative Cycle



If the distance of destination vertex > (distance of source vertex + weight between source and destination vertex):
Update distance of destination vertex to (distance of source vertex + weight between source and destination vertex)

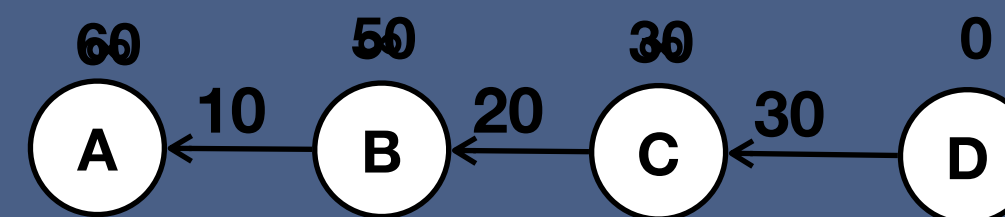
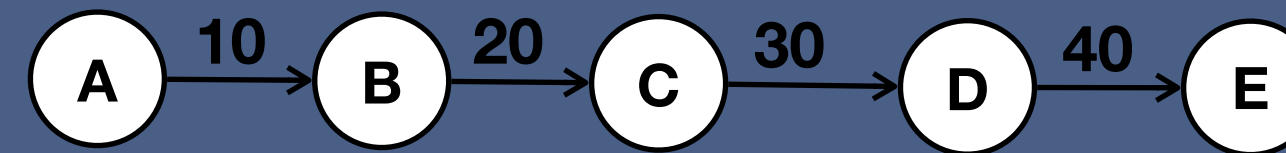
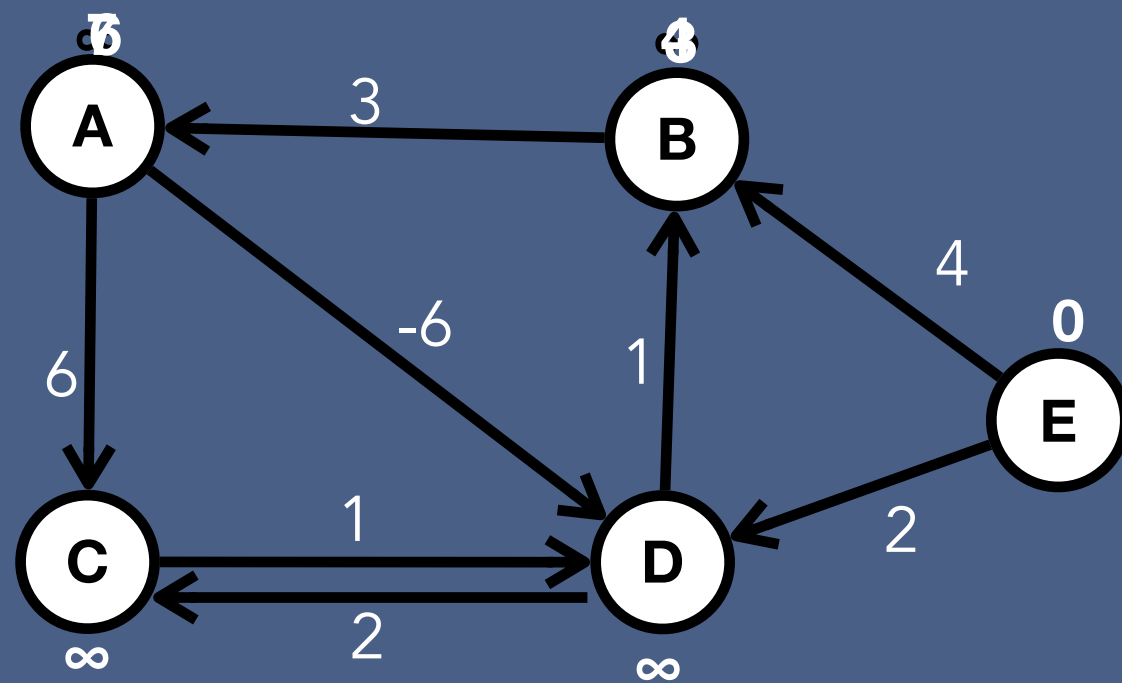
Edge	Weight
A->C	6
A->D	6
B->A	3
C->D	1
D->C	2
D->B	1
E->B	4
E->D	2

Final Solution		
Vertex	Distance from E	Path from E
A	6	E -> D -> B -> A
B	3	E -> D -> B
C	4	E -> D -> C
D	2	E -> D
E	0	0

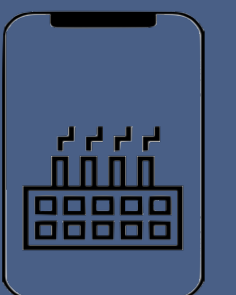


Why does Bellman Ford run V-1 times?

- If any node is achieved better distance in previous iteration, then that better distance is used to improve distance of other vertices
- Identify worst case graph that can be given to us

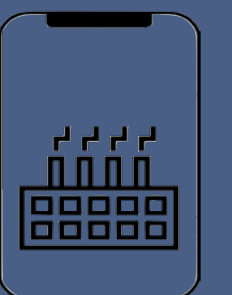


A → B
B → C
C → D



BFS vs Dijkstra vs Bellman Ford

Graph Type	BFS	Dijkstra	Bellman Ford
Unweighted - undirected	OK	OK	OK
Unweighted - directed	OK	OK	OK
Positive - weighted - undirected	X	OK	OK
Positive - weighted - directed	X	OK	OK
Negative - weighted - undirected	X	OK	OK
Negative - weighted - directed	X	OK	OK
Negative Cycle	X	X	OK



BFS vs Dijkstra vs Bellman Ford

	BFS	Dijkstra	Bellman Ford
Time complexity	$O(V^2)$	$O(V^2)$	$O(VE)$
Space complexity	$O(E)$	$O(V)$	$O(V)$
Implementation	Easy	Moderate	Moderate
Limitation	Not work for weighted graph	Not work for negative cycle	N/A
Unweighted graph	OK	OK	OK
	Use this as time complexity is good and easy to implement	Not use as implementation not easy	Not use as time complexity is bad
Weighted graph	X	OK	OK
	Not supported	Use as time complexity is better than Bellman	Not use as time complexity is bad
Negative Cycle	X	X	OK
	Not supported	Not supported	Use this as others not support

